

Tree Data Structure

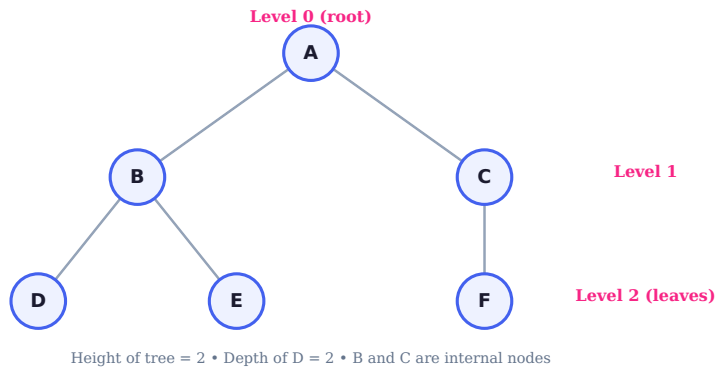
A complete visual guide — terminology, types, traversals, BST operations, balancing, heaps and tries — with diagrams for every concept.

- ▶ Terminology & Types
- ▶ Binary Search Trees (BST)
- ▶ Traversals: Preorder, Inorder, Postorder, Level-order
- ▶ Insertion & Deletion
- ▶ AVL Trees & Balancing
- ▶ Heaps & Tries
- ▶ Complexity Cheat-Sheet

What is a Tree?

A hierarchical, non-linear data structure

A **tree** is a collection of nodes connected by edges, where one node is designated the **root**, and every other node has exactly **one parent** and zero or more **children**. Trees have no cycles — they are a special kind of graph.



Basic tree terminology: root, internal nodes, leaves, and levels

Key Terms

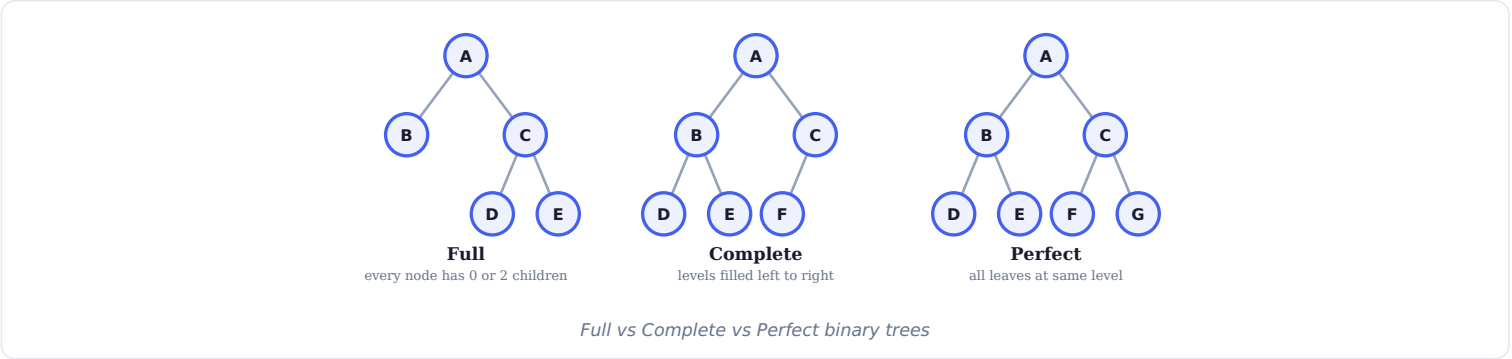
- **Root** — the topmost node (no parent)
- **Parent / Child** — a node directly connected one level up/down
- **Leaf** — a node with no children
- **Edge** — the connection between two nodes
- **Sibling** — nodes sharing the same parent

Measurements

- **Depth of a node** — number of edges from root to that node
- **Height of a node** — number of edges on the longest path down to a leaf
- **Height of tree** — height of the root
- **Degree** — number of children a node has

Types of Binary Trees

Shape-based classification you need for interviews

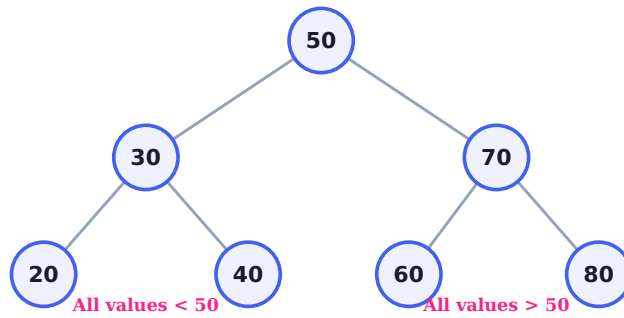


Type	Rule
Full (Proper)	Every node has 0 or exactly 2 children
Complete	All levels are fully filled except possibly the last, which fills left→right
Perfect	All internal nodes have 2 children AND all leaves are at the same level
Balanced	Height of left and right subtrees of every node differ by at most 1 (e.g. AVL)
Degenerate / Skewed	Each parent has only one child — behaves like a linked list

Binary Tree — each node has at most 2 children (left, right).
N-ary Tree — each node can have up to N children (e.g. a file-system directory tree).
Binary Search Tree (BST) — a binary tree with the ordering property (next section).

Binary Search Tree (BST)

Ordering property that enables fast search



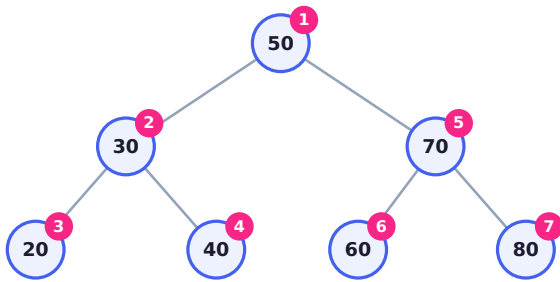
BST property: left subtree < node < right subtree, applied recursively

BST Property: for every node N , all values in its **left** subtree are $< N$, and all values in its **right** subtree are $> N$. This holds recursively for every subtree.

This property is what makes `search`, `insert`, and `delete` run in $O(\log n)$ on a balanced BST — at each step you eliminate one entire half of the remaining nodes, just like binary search on a sorted array.

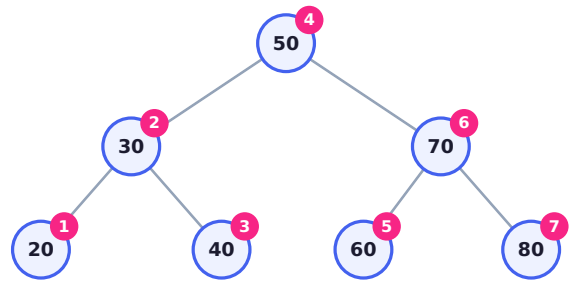
Tree Traversal Algorithms

The order in which nodes are visited — numbers on nodes show visit order



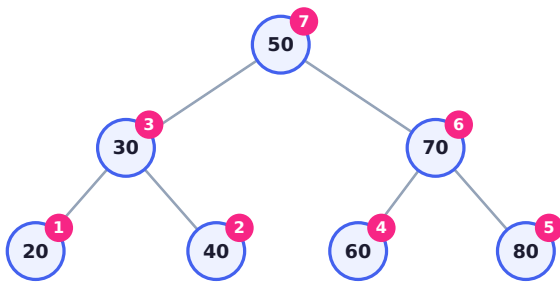
Preorder: Root → Left → Right

Result: 50, 30, 20, 40, 70, 60, 80



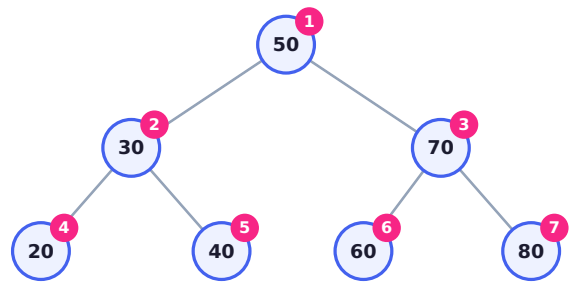
Inorder: Left → Root → Right

Result: 20, 30, 40, 50, 60, 70, 80 (sorted!)



Postorder: Left → Right → Root

Result: 20, 40, 30, 60, 80, 70, 50



Level-order (BFS): top row to bottom row, left to right

Result: 50, 30, 70, 20, 40, 60, 80

DFS-based: Preorder, Inorder, Postorder (use recursion or a stack) • **BFS-based:** Level-order (uses a queue)

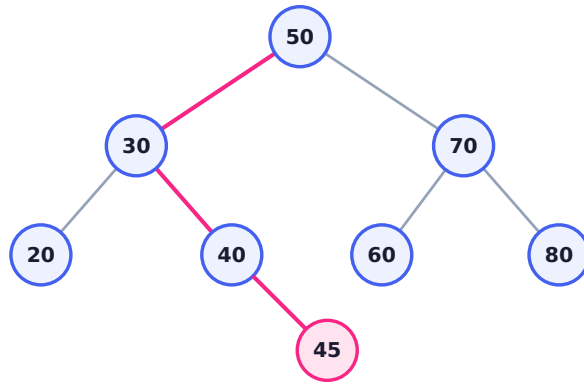
```
def inorder(node):
    if node is None:
        return
    inorder(node.left)
    visit(node) # process node
    inorder(node.right)
```

Tip: Inorder traversal of a BST always produces values in **sorted ascending order** — a very common interview fact.

BST Insertion & Deletion

Maintaining the BST property while modifying the tree

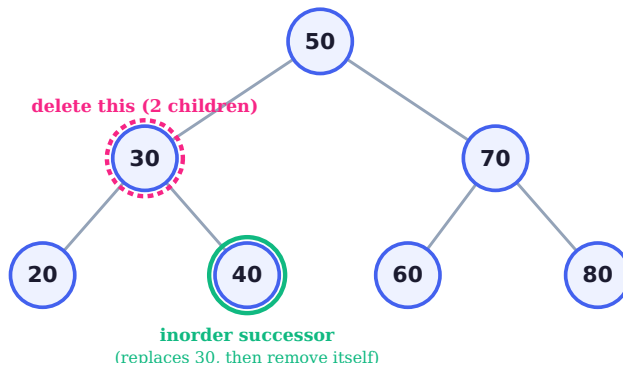
Insert 45



Compare 45 with each node, go right if greater / left if smaller, insert at the empty spot

Walk from the root: $45 < 50 \rightarrow$ go left to 30; $45 > 30 \rightarrow$ go right to 40; $45 > 40 \rightarrow$ 40 has no right child, so insert 45 there.

Delete a node with two children (30)



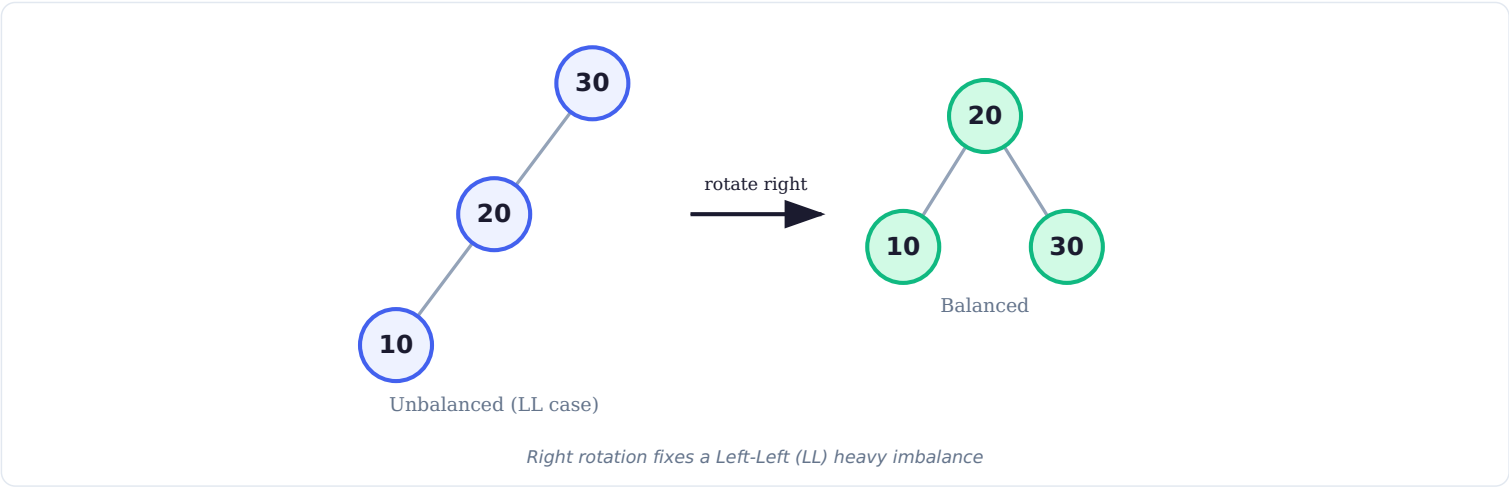
Replace the deleted node with its inorder successor (smallest value in right subtree), then delete the successor

- **Leaf node** — simply remove it
- **One child** — replace the node with its child
- **Two children** — replace with inorder successor (or predecessor), then delete that successor

Self-Balancing Trees: AVL

Keeping height $O(\log n)$ so operations stay fast

An **AVL tree** is a BST where, for every node, the **balance factor** (height of left subtree – height of right subtree) is always **-1, 0, or +1**. If an insertion/deletion breaks this, a **rotation** restores balance.



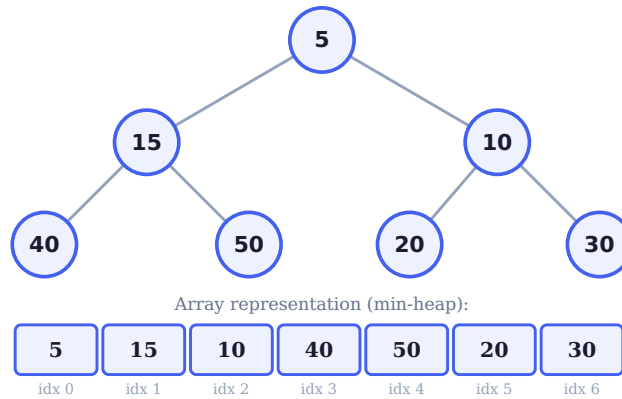
Imbalance Case	Fix
Left-Left (LL)	Single Right Rotation
Right-Right (RR)	Single Left Rotation
Left-Right (LR)	Left Rotation then Right Rotation
Right-Left (RL)	Right Rotation then Left Rotation

Other self-balancing trees: **Red-Black Trees** (used in C++ `map`, Java `TreeMap`), **B-Trees / B+ Trees** (used in databases & filesystems for disk-friendly wide branching).

Heaps & Tries

Specialized tree structures for priority and prefix problems

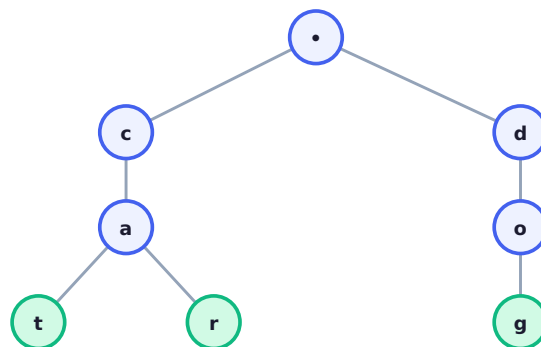
Binary Heap



Min-heap: every parent $\leq$ its children. Stored compactly as an array (no pointers needed).

For a node at array index i : parent = $(i-1)/2$, left child = $2i+1$, right child = $2i+2$. Used to implement **Priority Queues** and **Heap Sort**. Insert/extract-min run in $O(\log n)$.

Trie (Prefix Tree)



Green nodes = end of a word (cat, car, dog)

A trie storing "cat", "car", "dog" — shared prefixes share the same path

Each path from root to a marked node spells a word. Lookup and insert run in $O(L)$ where L is the word length — independent of how many words are stored. Used for autocomplete, spell-checkers, and IP routing tables.

Complexity & Applications

Everything on one page for quick revision

Structure	Search	Insert	Delete	Space
BST (average)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
BST (worst / skewed)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
AVL / Red-Black Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Binary Heap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Trie	$O(L)$	$O(L)$	$O(L)$	$O(\text{alphabet} \times n \times L)$

Where trees show up

File Systems folders & files hierarchy	Databases B/B+ trees for indexing	DOM HTML page structure
Routing tries for IP prefixes	Compilers abstract syntax trees	Scheduling heaps for priority queues